



COEP Technological University, Pune

Department of Computer Engineering

University Course Timetabling System

Semester VI B.Tech. Project Report

Academic Year: 2025–2026

Semester: VI

Guide: Abhijit A. M.

Abstract

University course timetabling is a well-known NP-Hard combinatorial optimization problem that involves the assignment of courses, instructors, student groups, and rooms to time slots while satisfying a complex set of hard and soft constraints. This project investigates the feasibility of deploying *UniTime*—an open-source, constraint-driven academic scheduling system—as a generalized timetabling solution for COEP Technological University. The university’s heterogeneous academic structure, which encompasses multiple departments, elective pathways, concurrent laboratory and lecture sessions, and varied instructional policies, presents significant scheduling challenges that manual or ad-hoc approaches cannot adequately address.

During the current semester, the project team evaluated the UniTime software framework with respect to system compatibility, constraint coverage, data integration, and algorithmic robustness. A local deployment was configured and populated with representative institutional data. The underlying solver employs an Iterative Forward Search (IFS) algorithm augmented with Conflict-based Statistics (CBS), which enforces all hard constraints while progressively minimizing soft-constraint violations. The results confirm that all relevant institutional constraints are captured within the UniTime constraint model. This report documents the system architecture, data preparation methodology, constraint mapping, algorithmic foundations, and the implementation strategy adopted for the project, along with identified directions for future development.

Contents

1	Introduction	4
2	Problem Definition	4
3	Overview of the UniTime System	5
3.1	System Description	5
3.2	Constraint Solver Library	5
3.3	Relevance to COEP	5
4	System Setup and Configuration	6
5	Data Preparation	6
5.1	Existing Institutional Data	6
5.2	Test Data Entry in UniTime	7
5.3	Curriculum Structure	7
5.4	Student Data Pipeline	7
5.5	Course Definition Interface	8
6	Constraint Analysis	8
7	Algorithmic Foundations	10
7.1	Iterative Forward Search	10
7.2	Pseudocode	10
7.3	Conflict-Based Statistics	11
7.4	Constraint Enforcement Guarantees	11
8	Implementation Details	12
8.1	Data Gathering and Formatting	12
8.1.1	Rooms, Instructors, and Courses	12
8.1.2	Student Data	12
8.1.3	Student Groups	12
8.2	Handling of Shared Laboratory Sessions	13
8.3	Lunch Break and Common Hour Configuration	13
9	Timetable Generation process	13
9.1	Input Data Validation	13
9.2	Executing the Automated Solver	14
9.3	Review and Interactive Adjustments	14
9.4	Saving and Committing the Timetable	15
9.5	Exporting the Solution to CSV	15

10 Future Scope	15
11 Conclusion	16

1 Introduction

Course timetabling at a university involves assigning a large number of classes—each with specific room, instructor, and timing requirements—to available slots in a weekly schedule, subject to a wide variety of hard and soft constraints. The problem is formally classified as NP-Hard, meaning that no polynomial-time exact algorithm is known, and practical solutions must rely on heuristic or metaheuristic methods [1].

COEP Technological University presents a particularly challenging instance of this problem. The institution offers undergraduate and postgraduate programmes across multiple engineering disciplines, with diverse academic structures that include core lectures, laboratory sessions, tutorials, elective courses, and cross-departmental cohort conflicts. These heterogeneous requirements demand a unified, extensible, and constraint-driven scheduling framework rather than isolated or manual approaches.

The objective of this project is to investigate, configure, and adapt *UniTime*—a comprehensive open-source academic scheduling system—to serve as the automated timetabling solution for the university. The work carried out during the previous semester established the constraint model and assessed problem feasibility. The present semester focused on the following tasks:

- Evaluating the architecture and feature set of the UniTime software.
- Configuring a local deployment and validating system compatibility.
- Integrating institutional data into the UniTime data model.
- Analyzing constraint handling by UniTime.
- Documenting the algorithmic foundations of the embedded solver.

2 Problem Definition

The core objective is to develop an automated course timetabling algorithm that adheres to all relevant institutional constraints of COEP Technological University. A comprehensive review of the literature confirms that *constraint programming* is the predominant methodology applied to this class of NP-Hard combinatorial problems, owing to its expressive power in modeling both mandatory (hard) and preferential (soft) constraints.

The *CPSolver* constraint programming library, which underlies the UniTime scheduling software, has been employed for this exact problem domain and is described in detail in the associated research literature [1]. During the preceding semester, the project team formally defined the relevant hard and soft constraints and assessed the theoretical feasibility of the timetabling problem as applied to this institution. The present phase extends that work by mapping the identified constraints onto the UniTime model and validating the mapping through controlled data experiments.

3 Overview of the UniTime System

3.1 System Description

UniTime is a comprehensive, open-source academic scheduling solution that has been deployed at large institutions worldwide, including Purdue University in Indiana (serving up to 40,000 students). The software is released under the Apache License, Version 2.0, is web-based, and is implemented in Java using modern enterprise technologies. It is a sponsored project of the Apereo Foundation and is in active use across institutions in the United States, Czech Republic, Pakistan, Croatia, Poland, Turkey, Peru, and Kuwait.

The system comprises four integrated components:

1. **Course Timetabling** – Scheduling of instructional offerings into rooms and time slots.
2. **Examination Timetabling** – Scheduling of mid-semester and end-semester examinations.
3. **Student Scheduling** – Assigning individual students to specific class sections based on enrolment preferences.
4. **Event Management** – Scheduling of non-curricular events within university spaces.

The software comprises over 500,000 lines of code, inclusive of the embedded constraint solver, and supports distributed data entry in multi-user environments. It is designed to be easily extended and customized for institution-specific requirements.

3.2 Constraint Solver Library

The embedded constraint solver is a publicly available library (GNU LGPL) based on a local search framework that incorporates constraint programming primitives. It is notable for winning two tracks of the International Timetabling Competition 2007 and reaching the final in all three tracks. The library is applicable to a broad class of constraint satisfaction and optimization problems and provides automated identification of inconsistencies and potential problems in input data.

3.3 Relevance to COEP

A primary concern in adapting UniTime for COEP Technological University was the handling of *batches*—groups of students who attend both laboratory and lecture sessions under the same instructional offering, potentially at different times or with different instructors. UniTime addresses this through a hierarchical mapping of *subparts* (lecture, laboratory, tutorial) within each instructional offering, each carrying independently configurable *Time Patterns*. This mechanism directly supports the scheduling requirements arising from batch-based laboratory and lecture structures.

4 System Setup and Configuration

The UniTime system was deployed on a Windows workstation following the official installation procedure. The required software stack consists of the following components:

1. **Java 21** – Runtime environment for the UniTime web application.
2. **MySQL 8.4** – Relational database backend for persistent storage of scheduling data.
3. **Apache Tomcat 9.0** – Java Servlet container hosting the UniTime web application.
4. **MySQL JDBC Driver** – Connector enabling communication between Tomcat and the MySQL database.
5. **UniTime Application** – Deployment of the UniTime `.war` file into the Tomcat webapps directory.

The installation was carried out in accordance with the official UniTime documentation¹. Following successful deployment, the system was verified to be accessible via the local web interface, and an administrative account was configured to facilitate data entry and solver execution.

5 Data Preparation

5.1 Existing Institutional Data

The project mentor provided access to the *Taasika* SQL database, which contains a representation of the university's existing timetable. This database was installed and examined to understand the current data structures used at the institution. This analysis informed the design of the data migration and transformation pipeline required for UniTime.

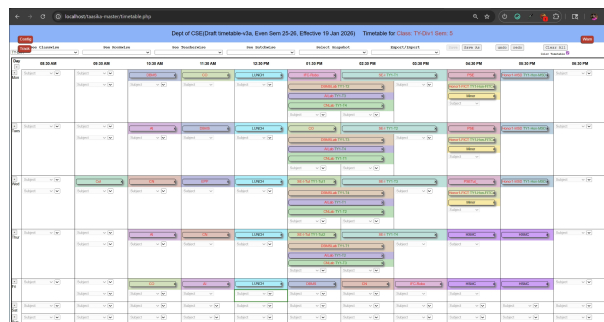


Figure 1: University timetable on taasika (TY-Div1)

¹<https://help.unitime.org/manuals/installation-windows>

5.2 Test Data Entry in UniTime

To understand and validate the UniTime data model, a new academic session was created within the system and populated with representative test data. The entities entered included:

- **Instructors** – Faculty members with associated availability and preference data.
- **Rooms** – Classrooms and laboratories with capacity, type, and feature metadata.
- **Instructional Offerings** – Course offerings comprising one or more subparts (lecture, laboratory, tutorial).
- **Time Patterns** – Templates defining permissible meeting day and duration combinations for each subpart.

Data entry was performed both manually via the web interface and through the formal XML import mechanism supported by UniTime [3]. The manual entry process provided insight into the constraint checking performed at the point of data entry, confirming that the system enforces constraint consistency incrementally during data input.

5.3 Curriculum Structure

The academic curriculum was modeled within UniTime using the following hierarchical structure, reflecting the actual organizational units of the university:

UniTime Entity	COEP Mapping
Academic Area	Engineering
Academic Classification	Semester 1 through 8; MTech Semester 1–2
Major	Computer Science and Engineering, Artificial Intelligence, etc.
Concentration	DSY (Department-Specific Elective)
Minor	MDMs (Minor Degree)
Degree	B.Tech., M.Tech.

5.4 Student Data Pipeline

Student data for choice-based course selection will be sourced from the university's MIS Portal. The required data fields are: student name, MIS number, date of birth, semester/branch/year, and selected course preferences. A transformation script will convert this data into the UniTime student XML import format², enabling automated bulk upload of student records and course selections.

²Schema reference: <https://www.unitime.org/interface/woebegonStudents.xml>

5.5 Course Definition Interface

A custom user interface is to be developed through which instructors can specify their lecture and laboratory availability and preferences. The submitted data will be programmatically transformed to generate all class instances for the academic semester, which will then be uploaded to UniTime via its XML import interface.

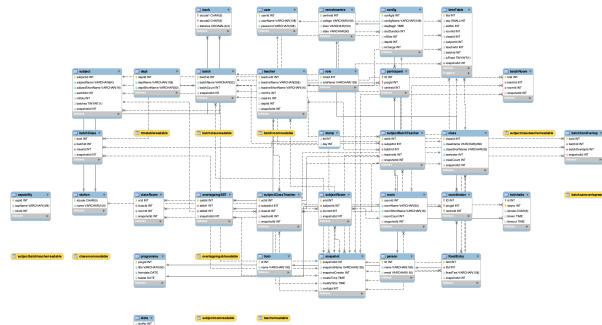


Figure 2: taasika schema

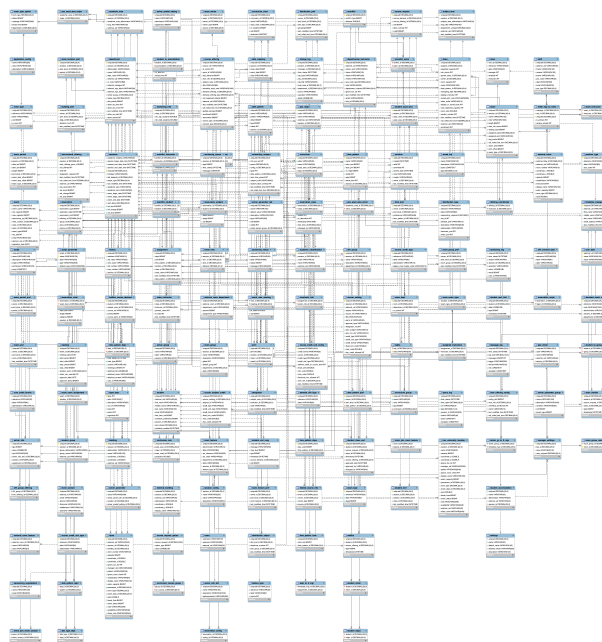


Figure 3: unitime schema

6 Constraint Analysis

A comprehensive set of scheduling constraints was identified in the prior semester and subsequently mapped onto the corresponding mechanisms available within UniTime. Table 1 summarizes the full set of constraints and their handling within the system.

Table 1: Institutional Scheduling Constraints and UniTime Handling

Constraint	Handling Mechanism in UniTime
Distance Between Rooms	Distance check integrated into the Room Detail configuration.
Preferred Labs and Lectures	Defined as subparts within Instructional Offerings, with independently configured Time Patterns per subpart.
Evenly Distributed Working Hours	Instructor working hours configured via preference settings; the solver prioritizes consecutiveness to minimize fragmentation.
Minimization of Schedule Holes (Students and Instructors)	Addressed by the <code>StudentMinimizeScheduleHoles</code> optimization criterion within the solver.
Consecutive Lectures for the Same Course	Enforced via Distribution Preferences; lectures of the same course scheduled within the same day are preferentially assigned to adjacent time slots.
Maximum Time Slots per Course per Day	Configured through Distribution Preferences to impose a per-day limit on time slot allocations for each course.
Instructor Load Management	Managed via Distribution Preferences limiting the number of consecutive (back-to-back) classes assigned to a given instructor.
Departmental Cohort Conflicts	Prevention of scheduling overlaps between courses intended for the same student cohort from different departments, enforced through Distribution Preferences.
Specialized Laboratory Allocation	Ensured by assigning laboratory sessions exclusively to rooms belonging to designated Room Groups, which encode hardware and equipment requirements.
Common University Hours (Lunch Break / Department Meetings)	Implemented by reserving designated time slots in the Time Pattern configuration, effectively blocking those periods from scheduling.

All constraints identified during the prior semester were verified to be expressible within the UniTime constraint model. No constraint required relaxation or external post-processing.

7 Algorithmic Foundations

7.1 Iterative Forward Search

The UniTime solver is based on the *Iterative Forward Search* (IFS) algorithm, as described by Rudová et al. (2011) [2]. IFS belongs to the family of local search methods; however, it differs from classical local search in that it operates over *feasible but potentially incomplete* solutions—solutions in which some classes may remain unassigned, but all hard constraints on assigned classes are strictly satisfied. This property renders intermediate solutions interpretable and actionable by human users, in contrast to complete but infeasible solutions produced by unconstrained search methods.

The algorithm operates iteratively from an initial (possibly empty) timetable ω and proceeds as follows at each step: a class variable is selected (typically an unassigned one, though assigned variables may be selected when refinement of soft constraints is required); a placement from its domain—comprising a time slot and a set of rooms—is chosen for assignment; any existing assignments that conflict with the selected placement under hard constraints are identified and removed (becoming unassigned); and the selected placement is assigned to the class. The best complete or partial timetable encountered during the search, denoted σ , is retained throughout and returned upon termination.

The search terminates either when a satisfactory timetable is found or when a pre-configured timeout is reached. Because of its iterative character, the solver can start, stop, or resume from any feasible timetable.

7.2 Pseudocode

The IFS algorithm is formally stated as follows:

Listing 1: Iterative Forward Search Algorithm (IFS)

```

1 function IFS(omega):
2     sigma = omega
3     while canContinue(omega) do:
4         v = selectVariable(omega)           // Select a class
5         d = selectValue(omega, v)           // Select a placement
6         gamma = hardConflicts(omega, v/d)   // Identify
            conflicting assignments
7         omega = omega \ gamma union {v/d}   // Remove conflicts,
            assign placement

```

```

8         if better(omega, sigma) then
9             sigma = omega                // Update best
              solution
10        end while
11        return sigma
12 end function

```

7.3 Conflict-Based Statistics

To prevent cycling and improve the quality of the final solution, the IFS algorithm incorporates a learning mechanism called *Conflict-based Statistics* (CBS), as developed by Müller et al. (2004). CBS maintains a data structure that records the frequency with which each pair of assignments has historically caused a hard conflict during the search:

$$\text{CBS}[A/a \rightarrow \neg B/b] = c_{ab}$$

This expression indicates that the assignation of class A to placement a has caused c_{ab} conflicts with the existing assignment B/b in previous iterations. During value selection, each candidate placement is penalized by the cumulative conflict frequency of all assignments it would displace, thereby guiding the search away from historically problematic regions of the solution space. When no complete solution is found, the content of the CBS structure provides diagnostic information identifying the hard constraints responsible for the infeasibility, which can be presented to the user to guide constraint relaxation.

7.4 Constraint Enforcement Guarantees

The solver framework enforces an invariant that hard constraints are never violated by any assignment that persists in the current timetable. Müller (2005) further demonstrates that arc consistency can be maintained within the framework throughout the search, and that IFS can be transformed into a complete search algorithm using dynamic backtracking. However, maintaining full arc consistency is computationally impractical for timetabling instances of realistic size, owing to excessive memory and CPU requirements. The solver therefore relies on the IFS heuristic in practice.

8 Implementation Details

8.1 Data Gathering and Formatting

8.1.1 Rooms, Instructors, and Courses

Structured data pertaining to rooms, instructors, and course offerings is collected from institutional sources and transformed from CSV format into UniTime-compatible XML via an automated conversion and upload script. This script enforces the required XML schema for each entity type and performs basic validation prior to upload.

8.1.2 Student Data

Student records and course preferences are retrieved from the university MIS Portal. A dedicated transformation script converts this data into the UniTime student import XML format, encoding each student's programme details and course selections as required by the system.

8.1.3 Student Groups

Student groups, which define cohorts sharing the same scheduling space (e.g., a batch of students attending a common laboratory), are defined through a rule-based configuration within UniTime. These groups are used to enforce cohort-level constraints and prevent conflicts across co-enrolled students.

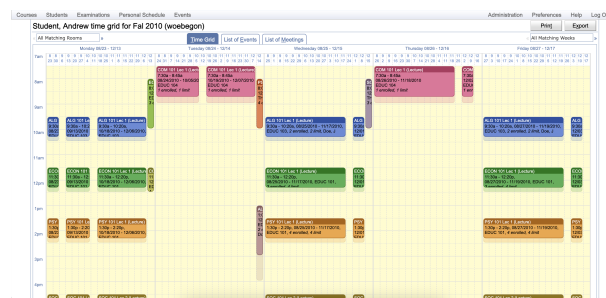


Figure 4: Student Group

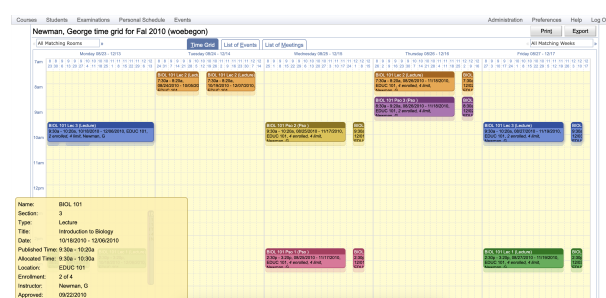


Figure 5: Courses Definition

8.2 Handling of Shared Laboratory Sessions

A specific edge case arises when two instructors are required to co-supervise a laboratory session for the same group of students, in the same room, at the same time. UniTime accommodates this scenario through the definition of multiple *lead instructors* on a single class. Additionally, courses that have multiple lecture or laboratory sections (e.g., an Operating Systems course with Lecture Section 1 assigned to Instructor A and Lecture Section 2 assigned to Instructor B) are modeled as separate class instances under the same instructional offering subpart.

Enrollment	Lead	Room	Mode	Days	Minutes per Week	Time Pattern	Room	Instructor	Time	Room	Cap
4	Inst	Full Term	100	2 x 50							
4	Inst	Full Term	100	1 x 100							
2	2	2	Inst	Full Term	100	2 x 50					
1	2	Inst	Full Term	100	2 x 50						
1	1	Inst	Full Term	100	1 x 100						
1	1	Inst	Full Term	100	1 x 100						
1	1	Inst	Full Term	100	1 x 100						
0	1	Inst	Full Term	100	1 x 100						

Figure 6: Two Instructors for a lab course case handling

8.3 Lunch Break and Common Hour Configuration

The enforcement of a universal lunch break or institution-wide common hour for department meetings is handled natively within UniTime. The relevant time slots are excluded from assignable Time Patterns in the *Administration* → *Solver* → *Configuration* settings, which also provides dedicated configuration parameters for both student and instructor lunch break preferences.

9 Timetable Generation process

9.1 Input Data Validation

Before initiating the automated solver, all foundational data—including instructional offerings, room capacities, instructor availability, and distribution preferences—must be checked for consistency.

- **Navigate to the Solver:** Access the Course Timetabling Solver screen via the sidebar menu.
- **Initial Check:** Set the solver configuration to ‘Check’ and click Load. This imports all prerequisite departmental data into the solver’s memory without starting the optimization sequence.

- **Review Diagnostics:** Upon reaching the "Awaiting commands..." status, refresh the screen. Review any system warnings to ensure 100% of the intended classes have successfully loaded and to identify any impossible hard constraints (e.g., an instructor booked for two mandatory overlapping classes).
- **Unload:** Once the data consistency is verified, click Unload to clear the check state and reset the solver.

9.2 Executing the Automated Solver

With verified data, the automated solver can begin constructing the schedule, balancing hard requirements against soft preferences.

- **Load for Optimization:** Change the solver configuration to 'Default' (or a specific departmental configuration) and click Load again.
- **Initialize Configuration:** Set the mode to 'Initial' to create a brand-new timetable from scratch.
- **Start the Solver:** Click Start. The solver will iterate through placements, attempting to minimize soft constraint violations (such as instructor back-to-back limits or campus travel times). For extensive departmental schedules, it is highly recommended to allow the solver to run until its predefined timeout limit (typically 30 minutes) to achieve the most optimized state.
- **Student Sectioning:** When the solver finishes, click Student Sectioning. This final computational phase reorganizes last-like student course demands to accurately minimize student enrollment conflicts based on the newly generated time and room placements.

9.3 Review and Interactive Adjustments

Automated generation rarely produces a flawless schedule on the first pass due to competing resource constraints. The Interactive Solver is used to bridge this gap and manually resolve edge cases.

- **Analyze Solution Properties:** Examine the "Best Timetable Found So Far" properties. If less than 100% of classes are assigned, navigate to the "Non-Assigned Classes" list. The profile for the solver can also be selected as Default(upto 30 mins) or Quick(for generating a timetable fast, in case of less constraints), and more.
- **Launch the Suggestions Screen:** Select an unassigned or problematically assigned class. UniTime will open a Suggestions pop-up, presenting alternative placement options.

- **Evaluate Perturbations:** The suggestions filter calculates the "ripple effect" of a proposed change. It will display options like room-only swaps or time-only swaps, and explicitly list any new student or faculty conflicts that the change would trigger.
- **Force Assignment:** Choose the optimal compromise from the suggested list and click Assign to manually enforce the placement.

9.4 Saving and Committing the Timetable

Once the schedule meets all operational requirements, it must be formalized in the system.

- **Save to Best:** During the interactive adjustment phase, periodically click Save To Best to temporarily hold your progress in the solver's memory.
- **Save As New:** To finalize the draft, click Save As New. This writes the generated timetable to the database as a saved solution.
- **Commit:** Navigate to the Saved Timetables screen, select your finalized schedule, and click Commit (alternatively, use Save Commit directly from the solver screen).

9.5 Exporting the Solution to CSV

For external auditing, reporting, and distribution, the finalized timetable can be exported into a flat-file format.

- **Direct Solver Export:** While the schedule is active on the Solver screen, click the Export Solution button. This instantly downloads the current assignments.
- **Saved Timetables Export:** Alternatively, from the Saved Timetables screen, select the committed timetable and click Export.
- **Output Data Structure:** The resulting CSV (Comma-Separated Values) file provides a structured matrix of the timetable. It includes comprehensive columns for subject areas, course numbers, section types, assigned instructors, allocated time slots, and room assignments, making it immediately ready for spreadsheet analysis or downstream system integration.

10 Future Scope

Several directions have been identified for extending this project beyond its current scope:

1. **Examination Scheduling.** The UniTime examination timetabling module can be activated to schedule mid-semester and end-semester examinations, with support for Teaching Assistants (TA1, TA2, etc.) as invigilators.

2. **Calendar Integration.** Since all students and instructors at COEP Technological University hold `@coeptech.ac.in` email accounts, the generated timetable can be automatically exported as iCalendar (`.ics`) events and pushed to individual Microsoft Outlook calendars, ensuring that each user's calendar reflects their personal schedule.
3. **University-Wide Deployment.** The current implementation is scoped to the Department of Computer Engineering. Future work should extend the system to encompass all departments and branches across the university.
4. **Automated Change Notification.** A notification system can be implemented to automatically dispatch email alerts to affected students and instructors whenever a timetable change is published.
5. **WhatsApp Group Automation.** An integration script could automatically create WhatsApp groups for each course and instructor, facilitating communication within each class section.

11 Conclusion

This project has demonstrated that the UniTime academic scheduling system is a viable and comprehensive solution for the automated course timetabling requirements of COEP Technological University. Through a systematic evaluation of the system's architecture, constraint model, algorithmic foundations, and data integration mechanisms, it has been established that UniTime captures all scheduling constraints identified by the project team without requiring any relaxation or external workarounds.

The Iterative Forward Search algorithm, augmented with Conflict-based Statistics, provides a robust and explainable optimization engine that enforces hard constraints absolutely while progressively improving soft-constraint satisfaction. The system's support for hierarchical curriculum modeling, batch-based laboratory and lecture structures, and distributed data entry makes it well-suited to the heterogeneous academic environment of the institution.

The groundwork laid during this semester—including the system deployment, constraint mapping, data pipeline design, and test data validation—provides a solid foundation for full-scale deployment in subsequent project phases.

References

- [1] Müller, T., Rudová, H., and Bartak, R. (2007). *Modeling and Solution of a Complex University Course Timetabling Problem*. Proceedings of the 6th International Conference on the Practice and Theory of Automated Timetabling (PATAT 2006). Available: <https://www.unitime.org/papers/patat07.pdf>

- [2] Müller, T. (2004). *ITC2007 Solver Description: A Hybrid Approach*. UniTime Technical Report. Available: <https://www.unitime.org/papers/lscs04.pdf>
- [3] UniTime Documentation. *Course Entry Manual*. Available: <https://help.unitime.org/manuals/courses-entry>